

Práctica Guiada de Qt Android + OpenCV

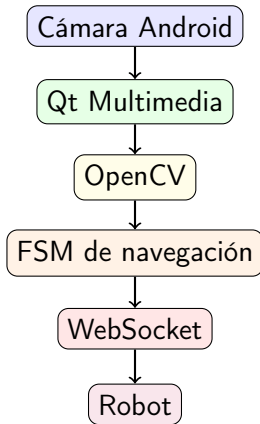
Diseño de Interfaces

Objetivo

El objetivo de esta práctica es construir una aplicación Android en Qt que:

- Capture video desde la cámara del teléfono.
- Procese imágenes usando OpenCV.
- Detecte líneas negras y azules.
- Implemente una FSM para navegación.
- Se comunique por WebSockets.
- Controle un robot móvil.

Diagrama



Objetivo

Este archivo define:

- módulos Qt utilizados
- compilación Android
- integración OpenCV
- librerías dinámicas

Configuración General del Archivo .pro

```
QT += core gui websockets multimedia

greaterThan(QT_MAJOR_VERSION, 4): QT += widgets

CONFIG += c++17

SOURCES += \
    main.cpp \
    mainwindow.cpp \
    worker.cpp

HEADERS += \
    mainwindow.h \
    worker.h

FORMS += \
    mainwindow.ui
```

Explicación de la Configuración General

Teoría

Esta sección configura:

- módulos Qt utilizados
- archivos fuente
- headers
- formularios UI
- estándar C++17

CONFIG += c++17

Activa características modernas de C++.

QT += websockets multimedia

Habilita:

- comunicación WebSocket
- cámara Android
- captura multimedia

Configuración Android y OpenCV

```
android {  
  
    ANDROID_PACKAGE_SOURCE_DIR = $$PWD/android  
  
    OPENCV_ANDROID = $$PWD/OpenCV-android-sdk/sdk/native  
  
    INCLUDEPATH += $$OPENCV_ANDROID/jni/include  
  
    LIBS += -L$$OPENCV_ANDROID/libs/arm64-v8a \  
            -lopencv_java4  
  
    ANDROID_EXTRA_LIBS += \  
        $$OPENCV_ANDROID/libs/arm64-v8a/libopencv_java4.so  
}
```

Teoría

Esta sección integra OpenCV en Android.

INCLUDEPATH

Permite acceder a los headers de OpenCV.

LIBS

Enlaza las bibliotecas dinámicas necesarias.

ANDROID_*EXTRA*_LIBS

Empaqueta las librerías dentro del APK Android.

Gracias a esta configuración, Qt puede usar OpenCV directamente en dispositivos Android.

Teoría

QT += websockets multimedia

Habilita:

- comunicación TCP/WebSocket
- captura multimedia
- cámara Android

CONFIG += c++17

Activa características modernas de C++.

ANDROID_EXTRA_LIBS

Permite empaquetar OpenCV dentro del APK.

Objetivo

El archivo main.cpp es el punto de entrada del programa.
Su función es:

- crear QApplication
- crear MainWindow
- iniciar el loop de eventos

Código Completo main.cpp

```
#include "mainwindow.h"
#include <QApplication>

int main(int argc, char *argv[])
{
    QApplication a(argc, argv);
    MainWindow w;
    w.show();
    return a.exec();
}
```

Teoría

QApplication

Administra:

- eventos
- widgets
- interacción con Android
- timers
- multimedia

a.exec()

Inicia el loop principal de Qt.

Objetivo

Define la clase principal del sistema.

Esta clase controla:

- cámara
- interfaz gráfica
- OpenCV
- WebSockets
- FSM del robot

Código Principal mainwindow.h

```
class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    explicit MainWindow(QWidget *parent = nullptr);
    ~MainWindow();

private:
    QMediaCaptureSession m_session;
    std::unique_ptr<QCamera> m_camera;
    std::unique_ptr<QVideoSink> m_sink;

    QThread m_workerThread;
    FrameWorker* m_worker = nullptr;

    QWebSocket *m_webSocket;
    QTimer m_reconnectTimer;
    QTimer m_pingTimer;
};
```

Teoría

QCamera

Controla la cámara física del dispositivo.

QVideoSink

Recibe frames de video.

QThread

Permite procesamiento paralelo.

QWebSocket

Permite enviar comandos al robot.

Objetivo

Separar el procesamiento de video de la interfaz gráfica.
Esto evita congelamientos.

Código Completo worker.h

```
class FrameWorker : public QObject {
    Q_OBJECT

public slots:
    void process(const QVideoFrame &frame);

signals:
    void ready(const QImage &img);
};
```

Teoría

Slots

Los slots permiten recibir señales.

Signals

Las señales permiten comunicar hilos.

Arquitectura producer-consumer

La cámara produce frames.

El worker los consume.

Objetivo

Convertir frames de Qt a OpenCV.
También normaliza el formato de imagen.

Código Principal worker.cpp

```
QImage img = frame.toImage();  
  
img = img.convertToFormat(QImage::Format_RGBA8888);  
  
cv::Mat rgba(img.height(), img.width(), CV_8UC4,  
             const_cast<uchar*>(img.bits()),  
             img.bytesPerLine());
```

Teoría

Qt utiliza QImage.

OpenCV utiliza cv::Mat.

Esta sección convierte entre ambos formatos.

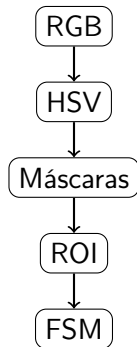
Objetivo

Implementar toda la lógica del sistema.

Incluye:

- procesamiento de imagen
- FSM
- control del robot
- comunicación WebSocket

Diagrama



Conversión RGB $\rightarrow$ HSV

```
cv::cvtColor(rgba, rgb, cv::COLOR_RGBA2RGB);  
cv::cvtColor(rgb, hsv, cv::COLOR_RGB2HSV);
```

Teoría

HSV facilita la segmentación por color.

Hue = color

Saturation = intensidad

Value = brillo


```
cv::inRange(  
    hsv,  
    cv::Scalar(0, 0, 0),  
    cv::Scalar(180, 255, 80),  
    maskNegro  
);
```

Teoría

La función `inRange` crea máscaras binarias.
Los píxeles dentro del rango se vuelven blancos.
Los demás se vuelven negros.

Región de Interés ROI

```
int roiH = 60;  
int roiY = (alto / 2) - (roiH / 2);  
  
cv::Rect roiRect(0, roiY, ancho, roiH);  
cv::Mat roi = binaria(roiRect);
```

Teoría

La ROI reduce el costo computacional.
Solo se analiza una franja horizontal.

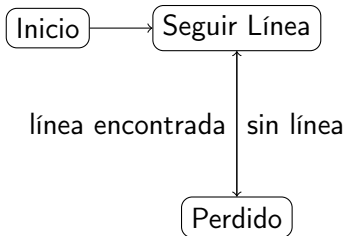
```
centroCamino = (xIzq + xDer) / 2;  
error = centroCamino - centroRobot;
```

Teoría

El error representa:

- desviación respecto al centro
- base para control del robot

Diagrama



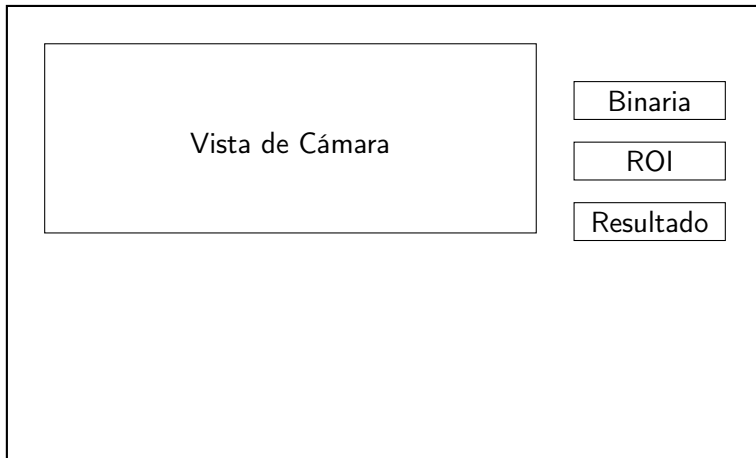
Objetivo

La interfaz permite:

- visualizar video
- seleccionar modos
- mostrar ROI
- monitorear estado

- QLabel
- QRadioButton
- QPushButton
- StatusBar

Diagrama



Objetivo

Enviar comandos en tiempo real al robot.

Ejemplo de JSON

```
{  
  "accion": "adelante",  
  "ms": 600  
}
```

Teoría

JSON permite enviar comandos estructurados.

Retos Propuestos

- 1 Agregar detección roja.
- 2 Implementar PID.
- 3 Agregar histograma HSV.
- 4 Detectar curvas.
- 5 Implementar grabación de video.
- 6 Añadir telemetría.
- 7 Mostrar FPS.
- 8 Agregar control manual.
- 9 Implementar joystick.
- 10 Añadir streaming.
- 11 Detectar obstáculos.
- 12 Agregar red neuronal.
- 13 Implementar mapas.
- 14 Optimizar OpenCV.
- 15 Agregar reconexión avanzada.

Resumen

En esta práctica aprendimos:

- Qt Android
- OpenCV
- procesamiento de imágenes
- visión artificial
- FSM
- WebSockets
- arquitectura multihilo

Pregunta

¿Cómo modificarías la FSM para que el robot pueda navegar en intersecciones complejas?